

CS3523 OS2 Assignment 1 Report

Extensions to xv6

Sri Sai Abhinav Reddy Boreddy CS24BTECH11014

1 Overview of the Assignment

The objective of this assignment was to understand and extend the xv6 operating system by implementing custom system calls and modifying core kernel data structures. The assignment focused on understanding system call wiring, kernel–user interaction, process relationships, and centralized accounting within the kernel. The work required careful modifications to existing xv6 subsystems while adhering to proper synchronization and locking discipline.

2 System Calls Implemented

The following system calls were implemented as part of the assignment:

- **hello()** – Prints the message ‘**Hello from the kernel!**’ to the xv6 console and always returns 0.
 - **getpid2()** – Returns the process identifier (PID) of the calling process. This system call was implemented independently of the existing **getpid()**.
 - **getppid()** – Returns the PID of the parent of the calling process. If the calling process has no parent, the system call returns **-1**.
 - **getnumchild()** – Returns the number of currently alive child processes of the calling process. Zombie processes are not counted.
 - **getsyscount()** – Returns the total number of system calls invoked by the calling process since its creation.
 - **getchildsyscount(int pid)** – Returns the system call count of a child process with the given PID if it is a child of the calling process, or **-1** otherwise.
-

3 Design Decisions and Assumptions

3.1 Locking Discipline

Access to parent–child relationships and the process table was synchronized using **wait_lock**, as these fields may be modified concurrently during process creation, exit, and wait operations. All traversals and accesses involving parent pointers were performed while holding this lock.

3.2 Process Table Traversal

For system calls requiring traversal of the process table, such as `getnumchild()` and `getchilddsyscount()`, the global process array was scanned linearly. This approach is consistent with xv6 design and sufficient for the scope of the assignment.

3.3 Failure Semantics

System calls return `-1` in cases where the requested operation is invalid, such as querying a non-child process or a process that no longer exists. No additional error codes were introduced.

3.4 Makefile and Testing Decisions

All user programs used for testing were added to the xv6 filesystem by updating the `UPROGS` list in the Makefile. Filenames were kept within the xv6 `DIRSIZ` limit to ensure correct filesystem creation.

4 Testing and Validation

The following user-level programs were used to validate the implemented system calls:

- `test.c` – Validates the `hello()` system call.
- `getpid2.c` – Compares the output of `getpid2()` with `getpid()`.
- `testppid.c` – Verifies correct parent PID retrieval using `getppid()`.
- `testgetnum.c` – Tests the correctness of `getnumchild()` across process creation and termination.
- `getsyscall.c` – Tests per-process system call accounting using `getsyscount()`.
- `getccount.c` – Validates `getchilddsyscount()` before and after the child process is reaped.

Each test program was executed from the xv6 shell and its output was compared against the expected behavior.

5 Results and Observations

All implemented system calls behaved as expected under normal execution. The system call accounting mechanism correctly counted all system calls, including those invoked internally by library functions. Parent-child relationships were accurately reflected, and zombie processes were handled correctly based on the semantics of each system call.

6 Screenshots for Evidence of Correctness

```

xv6 kernel is booting

hart 1 starting
hart 2 starting
init: starting sh
$ test
Hello from the kernel!
$ getpid2
The value of getpid = 4
The value of getpid2 = 4
$ testppid
my pid = 5
parent pid = 2
$ testgetnum
children before fork = 0
children after fork = 1
children after wait = 0
$ getsyscall
x
syscalls before = 3
syscalls after  = 7
difference = 4
$ getccount
c
child syscall count = 4
after wait = -1
$ █

```

Figure 1: Output of the `hello()` system call showing kernel message

7 Edge Cases and Handling

Potential edge cases were considered for each system call:

- Processes without parents correctly return `-1` in `getppid()`.
- Zombie processes are excluded from `getnumchild()` but allowed in `getchildsyscount()`.
- Invalid PIDs passed to `getchildsyscount()` correctly return `-1`.
- System call counters remain accurate across `fork`, `exit`, and `wait` operations.

8 Generative AI Use

Generative AI tools were used for:

- Understanding complex xv6 code segments, particularly those involving system call dispatch, process structures, and control flow.
- Identifying and reasoning about possible edge cases in process relationships and system call accounting.
- Assisting with LaTeX syntax and document structuring for the report.
- Locating and understanding resources required for downloading, configuring, and setting up development tools related to xv6.